Google Play Store Deployment Setup Guide

🎯 Overview

Your BusNStay app can now be automatically built and deployed to Google Play Store using GitHub Actions. This guide walks you through the setup process.

📋 Prerequisites

1. ☑ GitHub repository with Flutter code
2. ⏳ Google Play Developer Account ($25 one-time fee)
3. ⏳ Android App Signing Key (keystore)
4. ⏳ Play Store Service Account credentials

🔑 Step 1: Create Android Signing Key (Keystore)

Your Android app needs to be signed with a certificate. Generate it locally:

On Windows (PowerShell):

```
cd ~ # or your desired directory
keytool -genkey -v -keystore busnstay-key.jks -keyalg RSA -keysize 2048 -
validity 10000 -alias busnstay_key
```

On Mac/Linux:

```
keytool -genkey -v -keystore ~/busnstay-key.jks -keyalg RSA -keysize 2048 -
validity 10000 -alias busnstay_key
```

When prompted, enter:

```
Keystore password: [YOUR_STRONG_PASSWORD]
First and last name: Zwesta Empire Records
Organizational unit: BusNStay
Organization: Zwesta
City/Locality: Lusaka
State/Province: Lusaka
Country code: ZM
```

Save these values:

- Keystore file: `busnstay-key.jks`

 Copyright (c) Zwesta Empire All rights reserved.

- Keystore password: `[YOUR_STRONG_PASSWORD]`
- Key alias: `busnstay_key`
- Key password: `[YOUR_STRONG_PASSWORD]` (usually same as keystore)

🔐 Step 2: Encode Keystore for GitHub Secrets

Convert your keystore to Base64 for storage in GitHub:

Windows (PowerShell):

```
$keystore = [Convert]::ToBase64String([IO.File]::ReadAllBytes("$HOME\busnstay-key.jks"))
$keystore | Set-Clipboard
# Now paste in GitHub Secrets
```

Mac/Linux:

```
cat ~/busnstay-key.jks | base64 | pbcopy  # Mac
cat ~/busnstay-key.jks | base64 | xclip -selection clipboard  # Linux
```

🎮 Step 3: Google Play Console Setup

3.1 Create Play Store Account

1. Go to https://play.google.com/console/
2. Sign up with your Google account
3. Pay $25 one-time developer fee
4. Accept all agreements

3.2 Create App Project

1. **Create app** → Name: `BusNStay`
2. **App type**: App
3. **Category**: Food & Drink
4. **Default language**: English

3.3 Create Service Account

1. Go to **Setup** → **API Access**
2. Create a new service account on Google Cloud Console:
   - Project: Your Play Console project
   - Service account name: `busnstay-github`
   - Role: `Editor`
   - Create JSON key
3. Download the JSON file (keep safe!)

 Copyright (c) Zwesta Empire All rights reserved.

4. Grant permission in Play Console:
    - Go back to Play Console
    - Link the service account
    - Grant `Release to production` permission

🔒 Step 4: GitHub Secrets Configuration

Add these secrets to your GitHub repository:

1. Go to **Settings** → **Secrets and variables** → **Actions**
2. Click **New repository secret** for each:

Required Secrets:

1. ANDROID_KEYSTORE_BASE64

- Value: Your Base64-encoded keystore from Step 2
- Description: "Base64 encoded Android keystore file"

2. ANDROID_KEYSTORE_PASSWORD

- Value: Password you created for the keystore
- Description: "Password for Android keystore"

3. ANDROID_KEY_PASSWORD

- Value: Password for the key inside keystore (usually same)
- Description: "Password for Android key"

4. ANDROID_KEY_ALIAS

- Value: `busnstay_key`
- Description: "Alias of the key in keystore"

5. PLAY_STORE_SERVICE_ACCOUNT_JSON

- Value: Entire contents of the JSON file from Step 3.3 (raw JSON)
- Description: "Service account JSON for Play Store deployment"

Optional Secrets:

SLACK_WEBHOOK_URL (optional)

- For Slack notifications on deployment status
- Get from your Slack workspace → Apps → Incoming Webhooks

📝 Step 5: Update pubspec.yaml

Ensure your app identifier matches Google Play Store:

 Copyright (c) Zwesta Empire All rights reserved.

```
android:
  package: com.busnstay.app
```

Update version in `pubspec.yaml`:

```
version: 1.0.0+1
# Format: version+buildNumber
```

🚀 Step 6: Test the Workflow

Trigger Deployment:

1. **Create a Git tag** on main branch:

   ```
   git tag v1.0.0
   git push origin v1.0.0
   ```

2. **Watch GitHub Actions**:
   - Go to Actions tab in your repo
   - "Build & Deploy to Google Play Store" workflow starts
   - It will:
     - ☑ Build Flutter app
     - ☑ Create APK & App Bundle
     - ☑ Sign with your keystore
     - ☑ Upload to Play Store (Internal Testing track)
     - ☑ Create GitHub Release

3. **Manual Trigger** (optional):
   - Go to Actions
   - Select "Build & Deploy to Google Play Store"
   - Click "Run workflow"

📊 Deployment Tracks

The workflow deploys to **Internal Testing** track first:

Track	Visibility	Users	Purpose
Internal	Private	Team only	QA, internal testing
Closed (Beta)	Limited	1,000 testers	Beta testing
Open Testing	Limited	Public	Public beta

Copyright (c) Zwesta Empire All rights reserved.

Track	Visibility	Users	Purpose
Production	Public	Everyone	Live release

To Promote to Production:

1. Log into Play Console
2. Go to **Release** → **Internal testing**
3. Click the build
4. **Promote to Production**
5. Review & publish (takes 1-3 hours to go live)

🧪 Testing the Build Locally

Before deploying to Play Store, test locally:

```
# Build APK (for testing on device)
flutter build apk --release

# Build App Bundle (for Play Store submission)
flutter build appbundle --release

# Install APK on connected device
flutter install build/app/outputs/apk/release/app-release.apk
```

📱 What Gets Built

The workflow creates:

1. **APK** (`app-release.apk`)
   - Direct Android installation file
   - For manual testing or direct distribution
   - Size: ~50-100MB
2. **App Bundle** (`app-release.aab`)
   - Optimized for Play Store
   - Smaller downloads (dynamic delivery)
   - Size: ~30-50MB

☑️ Workflow Steps Explained

1. **Checkout** - Downloads your code
2. **Setup Java** - Installs Java 17 (required for Android)
3. **Setup Flutter** - Installs Flutter 3.8.1
4. **Get dependencies** - Runs `flutter pub get`
5. **Run tests** - Executes unit tests (optional)

 Copyright (c) Zwesta Empire All rights reserved.

6. **Build APK** - Creates APK file
7. **Build App Bundle** - Creates store-ready bundle
8. **Setup signing** - Decodes your keystore and configures signing
9. **Upload to Play Store** - Deploys to Internal Testing track
10. **Create Release** - Creates GitHub Release with artifacts
11. **Notify Slack** - Sends status to Slack (if configured)

🐛 Troubleshooting

"Invalid keystore"

- Verify Base64 encoding is correct
- Check keystore file is valid: `keytool -list -v -keystore ~/busnstay-key.jks`

"Service account not authorized"

- Verify service account has Play Console permissions
- Go to **Setup** → **User and permissions**
- Add service account with `Release to production` role

"Build failed: package name mismatch"

- Ensure `pubspec.yaml` has correct package: `com.busnstay.app`
- Verify it matches Play Store app ID

"Workflow didn't trigger"

- Check tag format: must be `v*` (e.g., `v1.0.0`)
- Verify workflow file is in `.github/workflows/play-store-deploy.yml`
- Push tag to GitHub: `git push origin v1.0.0`

📊 Version Numbers

Format: `version: X.Y.Z+buildNumber`

Increment strategy:

- **Major** (X): Breaking changes, big features
- **Minor** (Y): New features, backwards compatible
- **Patch** (Z): Bug fixes
- **Build** (buildNumber): Auto-increment for Play Store

Example progression:

```
1.0.0+1   -> Initial release
1.0.1+2   -> Bug fix
1.1.0+3   -> New features
2.0.0+4   -> Major rewrite
```

 Copyright (c) Zwesta Empire All rights reserved.

⏰ Timeout Notes

Play Store builds can take 10-20 minutes. GitHub Actions has a default 360-minute timeout, so you're safe.

🎯 Next Steps After Setup

1. ☑ Create keystore and add to GitHub Secrets
2. ☑ Create Play Store Developer Account
3. ☑ Create Service Account and credentials
4. ☑ Add all 5 secrets to GitHub
5. ☑ Push a version tag: `git tag v1.0.0 && git push origin v1.0.0`
6. ☑ Watch workflow run in GitHub Actions
7. ☑ Verify build in Play Console
8. ☑ Promote to Production when ready
9. ☑ Monitor ratings and reviews
10. ☑ Update app for new features/fixes

📞 Support Resources

- **Flutter Build**: https://docs.flutter.dev/deployment/android
- **Play Console**: https://support.google.com/googleplay/android-developer/
- **GitHub Actions**: https://docs.github.com/en/actions
- **Key Tool**: https://docs.oracle.com/javase/8/docs/technotes/tools/windows/keytool.html

💡 Pro Tips

1. **Test on device first** before committing and pushing tags
2. **Use semantic versioning** for clear version history
3. **Keep keystore file offline** (don't commit to git)
4. **Rotate service account credentials** annually
5. **Monitor crash reports** in Play Console
6. **Use internal testing track** for QA before promotion
7. **Set up beta testers** for feedback before production release

---

Status: ☑ Ready for Play Store deployment!

Once you complete these steps, every time you create a version tag (e.g., `git tag v1.0.0`), your app will automatically build and deploy to Play Store Internal Testing track.

Copyright (c) Zwesta Empire All rights reserved.